

Laboratório Especial

AUTHOR

Isabela Diniz Cia

Atividade

Ao longo desta semana, você deverá desenvolver código em R, utilizando-se de tidyverse, ggplot, leaflet, ggcal e outros pacotes disponibilizados em HTML Widgets para responder uma questão relacionada ao conjunto de dados flights. Para tanto, considere os seguintes fatos:

- Cias Aéreas (AIRLINE) possuem diferentes aeronaves (codificadas em TAIL_NUMBER);
- (Exemplo hipotético) No dia 01/01/2015, a aeronave N431WN viaja de BWI para JFK;
- (Exemplo hipotético) No dia 02/01/2015, a aeronave N431WN viaja de JFK para ORD;
- E assim sucessivamente...

No caso apontado acima, pode-se ver que a aeronave N431WN realizou um trajeto (provavelmente único) durante todo o ano de 2015. Nos dias exemplificados, o trajeto foi BWI-JFK-ORD.

Parte 1:

Crie uma função que:

- a. Receba um valor de TAIL_NUMBER (por exemplo, N431WN);
- b. Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip);
- c. Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima);
- d. O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do voo como espessura da linha que conecta os aeroportos envolvidos no trajeto);

Parte 2:

- a. Seu código deve ser bem comentado;
- b. Os nomes das variáveis devem ser esclarecedores (por exemplo, x é um nome ruim);
- c. Seu código será avaliado por 5 de seus colegas;
- d. A parte de avaliação (descrita em c.) acontecerá na semana de 23 a 27 de setembro (com entrega obrigatória no dia 27);

Observações:

1. A função terá o argumento tail_number e arquivo;
2. O argumento tail_number será o código identificador do avião de interesse (por ex., N431WN);

3. O argumento arquivo será o nome do arquivo com caminho completo (por ex., `file.path("dados", "flights.csv.zip")`);
4. A função se chamará `analisa_aeronave`;
5. O arquivo deve ser lido por partes;
6. O mapa deve utilizar as coordenadas exatas dos aeroportos envolvidos (presentes no arquivo `airports`);
7. A função deve retornar uma lista (nomeada) com dois valores: a tabela utilizada para o gráfico e o gráfico propriamente dito;
8. O Jupyter não será suficiente para a realização da tarefa; utilize o RStudio; o laboratório continuará disponível nos horários da aula;
9. O código deverá ser submetido via Moodle até o dia 23 de setembro, às 10 da manhã (em área exclusiva a ser disponibilizada oportunamente).

```
library(tidyverse)
```

```
— Attaching core tidyverse packages ————— tidyverse 2.0.0 —
```

```
✓ dplyr      1.2.1    ✓ readr      2.2.0
✓ forcats    1.0.1    ✓ stringr    1.6.0
✓ ggplot2    4.0.3    ✓ tibble     3.3.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
✓ purrr      1.2.2
```

```
— Conflicts ————— tidyverse_conflicts() —
```

```
✗ dplyr::filter() masks stats::filter()
```

```
✗ dplyr::lag() masks stats::lag()
```

```
! Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readr)
library(leaflet)

airports <- read_csv(
  "airports.csv",
  show_col_types = FALSE
)

# Função para analisar os trajetos de uma aeronave ----

analisa_aeronave <- function(tail_number, arquivo) {

  # Guarda o código da aeronave que será analisada
  aeronave <- tail_number

  # Processa cada parte do arquivo ----

  processa_voos <- function(input, pos) {

    input %>%
      filter(.data$TAIL_NUMBER == aeronave)

  }

  # Lê o arquivo em partes ----
```

```
voos <- read_csv_chunked(  
  arquivo,  
  callback = DataFrameCallback$new(processa_voos),  
  chunk_size = 100000,  
  show_col_types = FALSE  
)
```

```
# Cria uma variável de data e hora ----
```

```
voos_filtrados <- voos %>%  
  mutate(  
    data = as.Date(  
      paste(YEAR, MONTH, DAY, sep = "-")  
    ),  
  
    hora = sprintf(  
      "%04s",  
      SCHEDULED_DEPARTURE  
    ),  
  
    data_hora = as.POSIXct(  
      paste(  
        data,  
        substr(hora, 1, 2),  
        substr(hora, 3, 4)  
      ),  
      format = "%Y-%m-%d %H %M"  
    )  
  ) %>%  
  arrange(data_hora)
```

```
# Identifica as diferentes trajetórias ----
```

```
voos_com_seq <- voos_filtrados %>%  
  mutate(  
    sequencia_quebra = lag(DESTINATION_AIRPORT) !=  
      ORIGIN_AIRPORT,  
  
    sequencia_quebra = replace_na(  
      sequencia_quebra,  
      TRUE  
    ),  
  
    seq_group = cumsum(sequencia_quebra)  
  )
```

```
# Adiciona as coordenadas dos aeroportos ----
```

```
voos_com_coords <- voos_com_seq %>%  
  left_join(  
    airports %>%  
      rename(  
        lat_origin = LATITUDE,
```

```

        lon_origin = LONGITUDE
    ),
    by = c("ORIGIN_AIRPORT" = "IATA_CODE")
) %>%
left_join(
  airports %>%
    rename(
      lat_dest = LATITUDE,
      lon_dest = LONGITUDE
    ),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
) %>%
mutate(
  velocidade_media = DISTANCE / (AIR_TIME / 60),
  duracao_horas = AIR_TIME / 60
)

# Cria o mapa com cada sequência separada ----

# Define paleta de cores baseada em velocidade
cor_para_velocidade <- function(vel) {
  if (is.na(vel)) return("#808080")
  if (vel < 400) return("#d73027")      # Vermelho
  if (vel < 450) return("#fc8d59")      # Laranja
  if (vel < 500) return("#fee090")      # Amarelo
  if (vel < 550) return("#91bfdb")      # Azul claro
  return("#4575b4")                    # Azul escuro
}

mapa <- leaflet() %>%
  addTiles() %>%
  setView(lng = -95, lat = 38, zoom = 4)

# Para cada sequência conectada, desenha uma linha
for (seq_id in unique(voos_com_coords$seq_group)) {

  voos_seq <- voos_com_coords %>%
    filter(seq_group == seq_id) %>%
    arrange(data_hora)

  # Origem do primeiro voo
  df1 <- voos_seq %>%
    slice(1) %>%
    transmute(
      lat = lat_origin,
      lon = lon_origin,
      aeroporto = ORIGIN_AIRPORT,
      velocidade = velocidade_media
    )

  # Destinos em ordem
  df2 <- voos_seq %>%
    transmute(
      lat = lat_dest,

```

```
    lon = lon_dest,
    aeroporto = DESTINATION_AIRPORT,
    velocidade = velocidade_media
  )

# Junta
trajeto_seq <- bind_rows(df1, df2)

# Remove pontos sem coordenadas
trajeto_seq <- trajeto_seq %>%
  filter(
    !is.na(lat),
    !is.na(lon)
  )

# Calcula estatísticas da sequência
vel_media_seq <- voos_seq %>%
  summarise(vel = mean(velocidade_media, na.rm = TRUE)) %>%
  pull(vel)

distancia_total <- voos_seq %>%
  summarise(dist = sum(DISTANCE, na.rm = TRUE)) %>%
  pull(dist)

tempo_total <- voos_seq %>%
  summarise(tempo = sum(AIR_TIME, na.rm = TRUE)) %>%
  pull(tempo)

num_voos <- nrow(voos_seq)

# Define estilo da linha (espessura e cor)

espessura <- 2 + (vel_media_seq - 400) / 25
espessura <- max(2, min(10, espessura))

cor <- cor_para_velocidade(vel_media_seq)

# Desenha a linha
mapa <- mapa %>%
  addPolylines(
    data = trajeto_seq,
    lng = ~lon,
    lat = ~lat,
    weight = espessura,
    color = cor,
    opacity = 0.8,
    dashArray = "0",
    popup = paste(
      "<b>Sequência:</b> ", seq_id, "<br>",
      "<b>Número de voos:</b> ", num_voos, "<br>",
      "<b>Distância total:</b> ", round(distancia_total, 1), " milhas<br>",
      "<b>Tempo total de voo:</b> ", round(tempo_total, 1), " minutos<br>",
      "<b>Velocidade média:</b> ", round(vel_media_seq, 2), " mph"
    ),
  ),
```

```
      highlightOptions = highlightOptions(
        weight = espessura + 2,
        color = "black",
        opacity = 1
      )
    )
  )

# Adiciona marcadores nos aeroportos da sequência
for (i in 1:nrow(trajeto_seq)) {
  ponto <- trajeto_seq[i, ]

  # Ícone diferente para origem vs destinos
  if (i == 1) {
    icone <- "circle"
    cor_icone <- "green"
    label_texto <- paste(ponto$aeroporto, "(SAÍDA)")
  } else {
    icone <- "circle"
    cor_icone <- "blue"
    label_texto <- ponto$aeroporto
  }

  mapa <- mapa %>%
    addCircleMarkers(
      lng = ponto$lon,
      lat = ponto$lat,
      radius = 5,
      fillColor = cor_icone,
      color = "white",
      weight = 2,
      opacity = 0.8,
      fillOpacity = 0.8,
      popup = label_texto
    )
}

# Adiciona legenda ----

mapa <- mapa %>%
  addLegend(
    position = "bottomright",
    colors = c("#d73027", "#fc8d59", "#fee090", "#91bfdb", "#4575b4"),
    labels = c("< 400 mph", "400-450 mph", "450-500 mph", "500-550 mph", "> 550 mph"),
    title = "Velocidade Média",
    opacity = 0.8
  )

# Retorna a tabela e o mapa ----

return(
  list(
    tabela = voos_com_coords,
    mapa = mapa
  )
)
```

```

    )
  }
}

resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = "flights.csv"
)

```

Warning in validateCoords(lng, lat, funcName): Data contains 1 rows with either missing or invalid lat/lon values and will be ignored

Warning in validateCoords(lng, lat, funcName): Data contains 1 rows with either missing or invalid lat/lon values and will be ignored

```
resultado$tabela
```

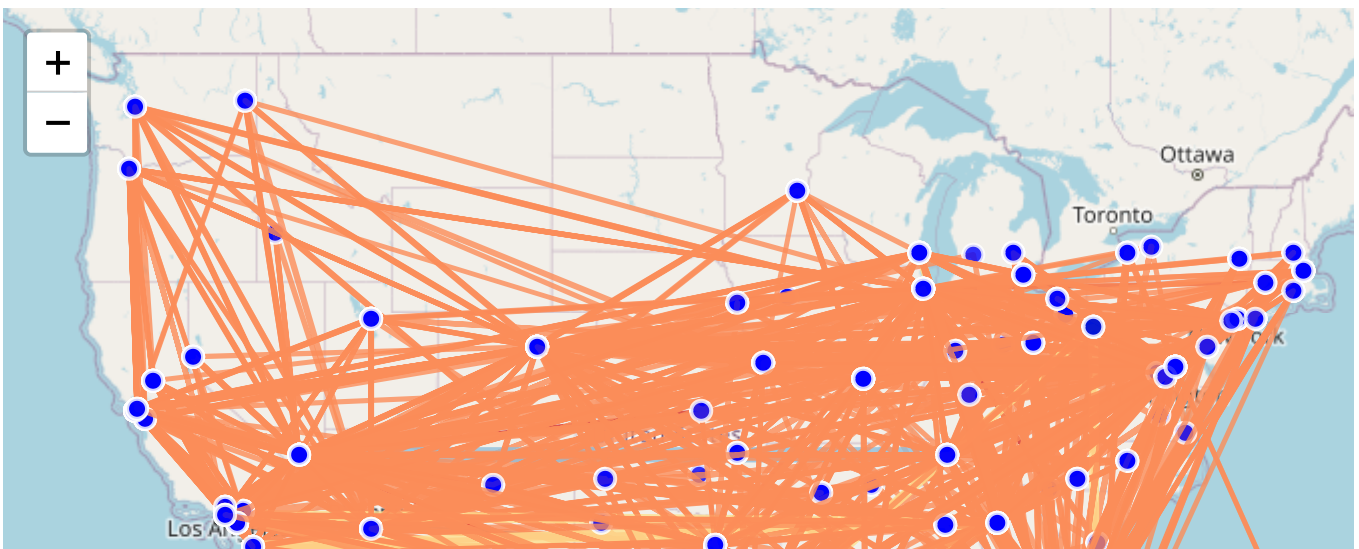
A tibble: 1,938 × 50

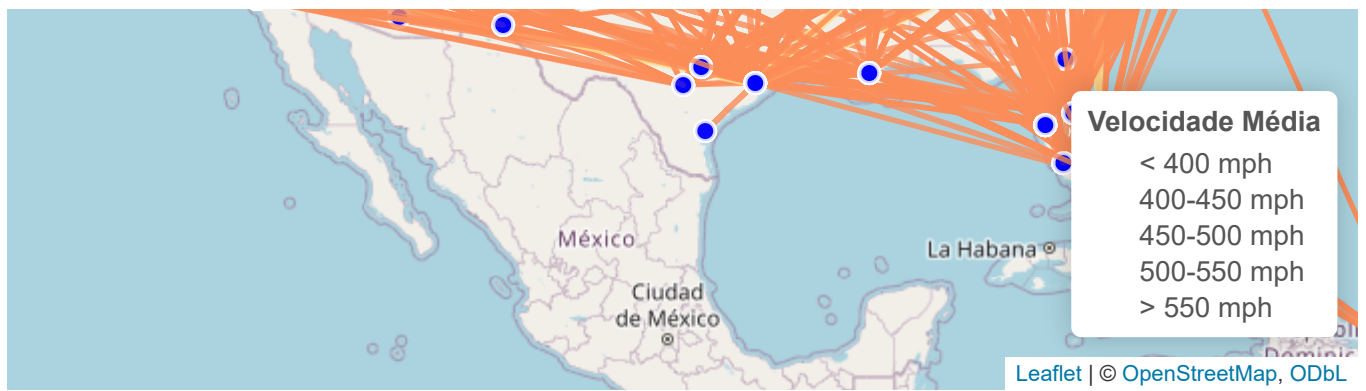
	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>
1	2015	1	1	4	WN	1762	N431WN
2	2015	1	1	4	WN	3564	N431WN
3	2015	1	1	4	WN	3564	N431WN
4	2015	1	1	4	WN	3564	N431WN
5	2015	1	2	5	WN	1793	N431WN
6	2015	1	2	5	WN	1793	N431WN
7	2015	1	2	5	WN	1793	N431WN
8	2015	1	2	5	WN	1793	N431WN
9	2015	1	2	5	WN	1793	N431WN
10	2015	1	2	5	WN	3144	N431WN

i 1,928 more rows

i 43 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
 # SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
 # TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
 # AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
 # SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
 # DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

```
resultado$mapa
```





Horário de conclusão

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 22:41:37 -03"
```